Graph Fraud Squad: Project Handoff

Hackathon: **Graph Hacks: Context for AI Agents** (WeMakeDevs x FalkorDB)
Page: https://www.wemakedevs.org/hackathons/falkordb
Tracks entered: **Track 1** (Agents That Act on Connected Data) and **Track 2** (Agent Memory and Coordination)
Dates: Oct 15-18, 2026. **Submissions close Sat Oct 18, 11:59 PM IST.**
Owner is a first-time hackathon participant on Windows (PowerShell), Python 3.12.

Hackathon rules to remember

- FalkorDB must be the **primary database** and central to the product (not decoration).
- The core FalkorDB implementation must be **new work done during the event**.
- Judges reward: multi-hop reasoning, explainable answers (show the path/query), persistent memory, per-user isolation, agent handoffs, concurrency.
- Live deployment preferred; otherwise full local setup instructions plus a working demo video.
- Coding assistants are allowed, but the owner must be able to explain the code.
- Side prize: best blog post. Submit a day early if possible.

What the project is

A **fraud investigation squad** built on FalkorDB. An LLM agent investigates flagged transactions by calling graph tools through MCP, **acts** (freezes accounts), and **remembers** past cases and lessons in a per-analyst memory graph. A Streamlit UI shows the live reasoning trace (with the Cypher behind each step), a graph picture, and memory/actions.

Architecture

```
Streamlit UI (app.py)
   -> agent_gemini.py   (LLM loop: Gemini, tool calling)
        -> MCP stdio server (mcp_server.py)
             -> graph_tools.py   -> FalkorDB graph "fraud"
             -> memory_tools.py  -> FalkorDB graphs "memory_<analyst>"
```

Fraud graph (`fraud`)

- Nodes: `Account{id,name,age_days,frozen?,freeze_reason?}` , `Device{id}` , `Merchant{id,name,category}` , `Transaction{id,amount,flagged,day}`

- Edges: `(Account)-[:USED]->(Device)`, `(Account)-[:MADE]->(Transaction)-[:TO]->(Merchant)`
- Seed data: ~200 normal accounts, 30 merchants, 1000 normal transactions, a few innocent shared devices, **3 hidden fraud rings** (R0_, *R1_*, R2_*; shared devices, shell merchants MS0/MS1/MS2), and an account `BRIDGE` that links ring 0 and ring 1 (so R0+R1 are really one network; R2 is separate).
- Flagged transactions: T1001, T1031, T1061.

Memory graph (`memory_<analyst>`, one per analyst = isolation)

- `(Case{id,summary,verdict,created})-[:INVOLVES]->(Entity{id})`
- `Lesson{rule,uses,created}` (use counter increments on repeat)

MCP tools (mcp_server.py)

Graph: `get_flagged_transactions`, `get_account_profile`, `find_linked_accounts`, `shortest_path`, `get_merchant_exposure`, `get_account_risk`
Actions: `freeze_account`, `get_frozen_accounts`
Memory: `recall_similar_cases`, `save_case`, `get_lessons`, `save_lesson`
Every graph tool returns `{"data":..., "cypher":...}` so the UI can show the query behind each answer.

Files (all in `graph-hacks/`)

File	Purpose
`seed.py`	Builds the `fraud` graph with synthetic data and rings. Safe to re-run (deletes and rebuilds).
`graph_tools.py`	Graph query functions and actions. `python graph_tools.py` runs a manual test of all read tools.
`memory_tools.py`	Per-analyst memory (cases, lessons).
`mcp_server.py`	Exposes all tools via FastMCP over stdio. Must never `print()` (stdout is the protocol).
`agent_gemini.py`	Gemini agent with a manual tool loop, model fallback and retries. `investigate(question, analyst, on_step)` is used by the UI.
`agent.py`	Claude version of the agent. **Stale**: lacks the analyst env fix, `on_step`, and the memory/action prompt. Update only if switching to Claude.
`app.py`	Streamlit UI (tabs: Report, Reasoning trace, Graph, Memory & actions).

File	Purpose
`requirements.txt`, `.gitignore`	Dependencies (should pin `mcp<2`); ignore `venv/`, `__pycache__/`, `.env`.

How to run (after any restart)

1. Docker Desktop open, FalkorDB container running (`docker ps -a`; `docker start <name>` if exited). Ports 6379 (DB) and 3000 (Browser UI).
   - Create: `docker run -d --name falkordb -p 6379:6379 -p 3000:3000 -v falkordb_data:/var/lib/falkordb/data falkordb/falkordb:latest`
2. `cd C:\Users\rajku\OneDrive\Desktop\falkorDB\graph-hacks`
3. `venv\Scripts\activate`
4. `$env:GEMINI_API_KEY="..."` (never commit keys; set per terminal)
5. If data is missing: `python seed.py`
6. `streamlit run app.py` (opens http://localhost:8501)

PowerShell notes: `&&` is not valid in this PowerShell version; run commands on separate lines.

Version and environment gotchas (learned the hard way)

- **Pin `mcp<2`.** mcp 2.x renamed `FastMCP` to `MCPServer` and changed APIs; the code targets 1.x.
- Passing the MCP session directly to the Gemini SDK fails (`cannot pickle _asyncio.Future`). The code uses a manual tool loop with converted function declarations instead.
- **MCP's stdio launcher does not forward most env vars** to the server subprocess. The analyst name is passed explicitly via `StdioServerParameters(env={"ANALYST": ...})`.
- Gemini models: `gemini-2.5-flash` is retired (shutdown Oct 16, 2026). `gemini-3.8-flash` and `gemini-3.7-flash` returned repeated 503 "high demand". **`gemini-3.5-flash-lite` worked** and is the default. `agent_gemini.py` cycles through fallbacks (`3.5-flash-lite`, `3.1-flash-lite`, `3.6-flash`) and retries. Check Google AI Studio for current names.
- The project folder lives in OneDrive; if odd file errors appear, move it to a plain path like `C:\projects\graph-hacks`.
- Switching Gemini models mid-conversation could cause history/thought-signature errors (untested).

Status

Verified working (by the owner):

- FalkorDB running; Browser works at localhost:3000.

- `seed.py` output: 216 accounts, 206 devices, 33 merchants, 1092 transactions, 2409 relationships.
- `graph_tools.py` manual test (flagged txns, linked accounts, shortest path R0_1 -> R1_2 via BRIDGE, merchant exposure, risk scores).
- `agent_gemini.py` end-to-end (before memory/action tools were added): correctly found R0+R1 as one network via BRIDGE and R2 as separate.

Written but NOT yet confirmed working:

- Memory tools (`save_case` , `recall_similar_cases` , `save_lesson` , `get_lessons`) and whether the agent actually calls them.
- Action tools (`freeze_account`) and the new system prompt.
- Analyst env fix (isolation test with analyst "bob" should show empty memory).
- `app.py` (all four tabs, pyvis graph drawing).
- `requirements.txt` pinning, and whether the repo was pushed to GitHub.

Still to do (priority order)

1. **Verify and debug** the memory + action + UI path: run the agent twice as `alice` (second run should call `recall_similar_cases` and not re-freeze), then as `bob` (empty memory). Fix errors; tighten the prompt if the small model skips `save_case` / `save_lesson` .
2. **Commit and push to GitHub** (public repo). Pin dependencies (`pip freeze > requirements.txt` , ensure `mcp<2`).
3. **Strengthen Track 2** (currently one agent with memory):
   - Implement real **multi-agent roles with handoff through the graph**: Triage agent writes an open `Case` node; Investigator agent reads it and closes it; optional Reporter agent. Show the handoff in the UI.
   - Show **isolation** (alice vs bob) and, if time allows, a **concurrency** demo (two analysts running at once).
4. **Strengthen Track 1:** use real **graph algorithms** (e.g. connected components / PageRank via FalkorDB's algorithm procedures; check docs.falkordb.com for exact syntax) as an additional MCP tool, so the graph is visibly central. Consider human-approval before `freeze_account` for realism.
5. **Deploy a live demo:** hosted FalkorDB (check FalkorDB Cloud options) plus Streamlit Community Cloud or Render. Store keys in the host's secrets, never in the repo. The MCP server is launched as a subprocess, which should work on those hosts but is untested. Fallback: local setup instructions plus the demo video.
6. **README:** problem, architecture diagram, track mapping (Track 1 and Track 2), setup steps, why a graph (not SQL), screenshots.
7. **Demo video (2-3 min):** (a) flagged transactions, (b) agent investigates live with tool calls and Cypher, (c) graph view showing BRIDGE joining rings, (d) agent freezes accounts, (e) restart, run again, show memory recall, (f) switch analyst to bob to show isolation.

8. **Blog post** (side prize): what was built, why FalkorDB, what was learned (include the gotchas above).
9. **Submit by Sat evening IST** (aim for about 8-9 PM, not 11:59 PM). Check the submission form for how to select both tracks, and re-read the Rules page for extra requirements.

Reset commands

- Unfreeze accounts (Cypher on graph `fraud`): `MATCH (a:Account) REMOVE a.frozen, a.freeze_reason`
- Wipe a memory graph (Cypher on `memory_alice`): `MATCH (n) DETACH DELETE n`
- Rebuild everything: `python seed.py`

Starter prompt for an AI assistant in VS Code

> I'm in a hackathon (Graph Hacks, FalkorDB) with a deadline of Sat Oct 18, 11:59 PM IST. Read `PROJECT_HANDOFF.md` in this repo first. The project is a fraud-investigation agent on FalkorDB with an MCP tool server, Gemini agent, per-analyst memory graphs and a Streamlit UI. I need to verify the memory/action/UI path, then work through the "Still to do" list in priority order. I'm a beginner on Windows PowerShell, so give exact commands, explain what you change, and keep changes small. Start by helping me run `streamlit run app.py` and debug anything that fails.